

Función isupper()

La función `any(c.isupper() for c in txtIn)` en Python verifica si **algún carácter en la cadena texto es una letra mayúscula**.

Desglose paso a paso:

1. **for c in txtIn**: Esto recorre cada carácter `c` de la cadena `txtIn`.
2. **c.isupper()**: Es un método de Python que devuelve `True` si el carácter `c` es una letra en mayúscula (como `'A'`, `'Z'`, etc.), y `False` en caso contrario.
3. **(c.isupper() for c in txtIn)**: Esto es un **generador**, que produce una secuencia de valores `True` o `False` dependiendo de si cada carácter es mayúscula.
4. **any(...)**: La función `any()` devuelve `True` si **al menos uno** de los elementos del generador es `True`. Si **ninguno** es `True` (es decir, no hay letras mayúsculas), devuelve `False`.

Ejemplo:

```
txtIn = "abc123"
```

```
resultado = any(c.isupper() for c in txtIn)
```

```
print(resultado) # False
```

```
txtIn = "abcD123"
```

```
resultado = any(c.isupper() for c in txtIn)
```

```
print(resultado) # True
```

Función isdigit()

La expresión `any(c.isdigit() for c in txtIn)` en Python verifica si **algún carácter en la cadena txtIn es un dígito numérico (del 0 al 9)**.

Desglose paso a paso:

1. **for c in txtIn**: Recorre cada carácter `c` en la cadena `txtIn`.

2. **c.isdigit()**: Este método devuelve True si el carácter c es un **dígito decimal** (como '0', '1', ..., '9'). Si no lo es, devuelve False.
3. **(c.isdigit() for c in txtIn)**: Esto crea un **generador** que produce una secuencia de True o False dependiendo de si cada carácter es un dígito.
4. **any(...)**: Devuelve True si **al menos uno** de los elementos del generador es True, es decir, si **hay al menos un número** en la cadena. Si no hay ninguno, devuelve False.

Ejemplo:

```
txtIn = "HolaMundo"
```

```
resultado = any(c.isdigit() for c in txtIn)
```

```
print(resultado) # False
```

```
txtIn = "Hola123"
```

```
resultado = any(c.isdigit() for c in txtIn)
```

```
print(resultado) # True
```

Función isalpha()

La expresión `cadena.isalpha()` en Python verifica si **todos los caracteres de la cadena** son **letras del alfabeto** y **no está vacía**.

Detalles clave:

- **Devuelve True** si:
 - Todos los caracteres son **letras** (A–Z o a–z, también incluye letras de otros alfabetos como á, ñ, etc.).
 - La cadena **no está vacía**.
- **Devuelve False** si:
 - Hay **espacios, números, símbolos** o **caracteres vacíos** en la cadena.

Ejemplos:

```
"Hola".isalpha()    # True
```

```
"HolaMundo".isalpha() # True
```

```
"Hola123".isalpha()  # False (tiene números)
```

```
"Hola Mundo".isalpha() # False (tiene espacio)
```

```
"".isalpha()        # False (cadena vacía)
```

Función strip()

La expresión `txtIn.strip()` en Python se utiliza para **eliminar los espacios en blanco al principio y al final** de una cadena `txtIn`.

¿Qué elimina exactamente?

`strip()` elimina:

- Espacios (" ")
- Tabulaciones (\t)
- Saltos de línea (\n)
- Otros caracteres en blanco (como \r)

Por defecto, no elimina caracteres **dentro** de la cadena, solo en los **bordes** (inicio y fin).

Ejemplos:

```
txtln = "  Hola mundo  "
print(txtln.strip()) # "Hola mundo"
```

```
txtln = "\n\t Texto con espacios y tabulaciones \t\n"
print(txtln.strip()) # "Texto con espacios y tabulaciones"
```

```
txtln = "SinEspacios"
print(txtln.strip()) # "SinEspacios" (no cambia nada)
```

```
txtln = " "
print(txtln.strip()) # "" (cadena vacía)
```

Función zfill ()

La función `zfill()` **rellena una cadena (str) con ceros (0) a la izquierda** hasta alcanzar una longitud especificada.

◆ Sintaxis:

```
cadena.zfill(ancho)
```

- **cadena:** la cadena de texto a la que se le quiere aplicar el relleno.
- **ancho:** longitud total deseada de la cadena resultante.
Si la longitud de la cadena original es mayor o igual al ancho, **no se agrega nada**.

◇ Ejemplos:

```
print("42".zfill(5))      # '00042'
print("7".zfill(3))       # '007'
```

```
print("python".zfill(10)) # '0000python'  
print("-5".zfill(4))      # '-005'
```

✅ Nota importante:

- Si la cadena comienza con un **signo positivo (+)** o **negativo (-)**, los ceros se agregan **después del signo**:

```
print("-42".zfill(5))  # '-0042'  
print("+3".zfill(4))   # '+003'
```

◇ ¿Para qué se usa comúnmente?

- Para formatear números con ceros a la izquierda (como en códigos, IDs o fechas).
- Para generar salidas con longitud uniforme.